From Karplus-Strong to waveguide synthesis using logue-sdk

KORG Inc.

Greetings!

- Shijie Xia 夏世傑
  - software engineer at KORG Inc.
  - works on embedded systems
- Davis Sprague
  - software engineer at KORG Inc.
  - works on digital signal processing (DSP)
  - KORG Collection (audio plugins), opsix, and microKORG2

What is logue-sdk?

an SDK to write your own audio oscillators/effects that run on KORG hardware

developed by Etienne Noreau-Hebert

- 2018: prologue
- 2019: NTS-1, minilogue xd
- 2022: drumlogue
- 2024: NTS-1 mkII, NTS-3
- 2025: microKORG firmware 2.0 adds logue-sdk support

How it works

- logue-sdk uses GNU Arm Embedded Toolchain (arm-none-eabi-gcc) to cross-compile your DSP code to the target product's CPU architecture
  - NTS-1, minilogue-xd, and prologue: ARM Cortex-M4
  - drumlogue, microKORG2: NXP i.mx6ulz (ARM Cortex-A7)
  - NTS-1 mkII and NTS-3: STM32H725 (ARM Cortex-M7)
- compiled binary (.elf, "unit") do not work across different CPU architectures
  - SDK 1.0 products (NTS-1, minilogue-xd, and prologue) do have binary compatibility
  - support for more hardwares have been added since
  - different hardwares may have slightly different APIs
- not a universal plugin framework like JUCE

Developer community

- Sinevibes
  - high quality, well-tested and reasonably priced fx suite
- hundreds of open source and paid projects
- a Discord of over 100 developers
- recently reached over 1k stars on GitHub

Today’s workshop

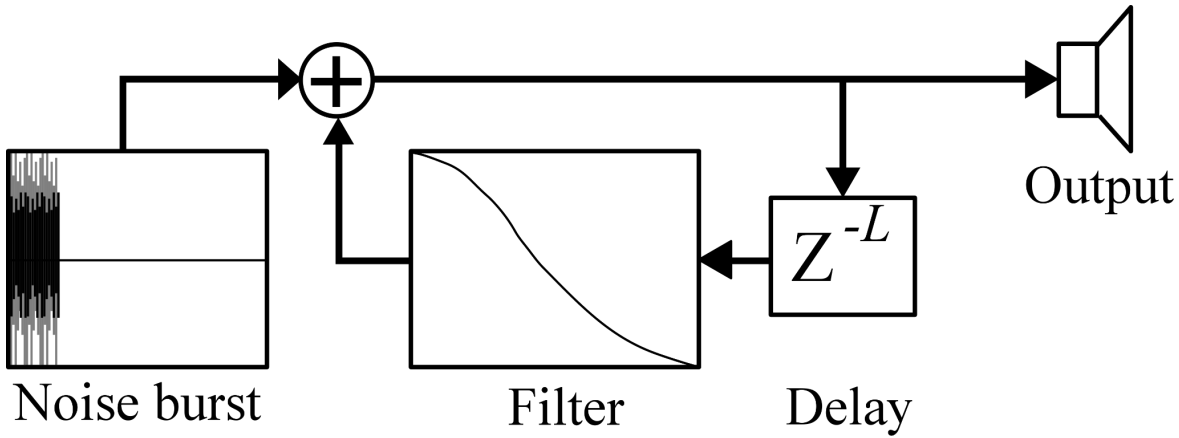
- Develop an oscillator based on the classic Karplus-Strong algorithm that works on NTS-1 mkII and NTS-3
- Github repo
  - https://github.com/xiashj24/pluck_logue_sdk
  - https://github.com/xiashj24/pluck_logue_sdk_nts3
- go through the implementation step by step (~1h)
  - using a web assembly based development environment
- hands on with hardware (~1h)
- Q&A session (~0.5h)

Karplus-Strong algorithm

- discovered by Kevin Karplus and Alex Strong, who were computer science students at Stanford, while trying out wavetable synthesis algorithms on an 8-bit computer
- David Jaffe - Silicon Valley Breakdown (1982)

Karplus-Strong algorithm

3 things to make a plucked string sound: noise, delayline, low-pass filter



Getting started with a dummy-osc project

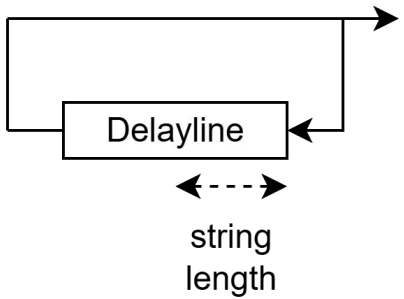
- how to setup the web assembly based simulator
  - https://github.com/korginc/logue-sdk/tree/main/websim
- the simulator calls the API of the DSP class in the same way as the NTS-1 mkII (mostly)

[1] use pitch info from note on

- pitch is passed to NTS-1 oscillator via the setPitch function
- this is sufficient for oscillators that are basically “drones”, e.g. wavetable
- to make the oscillator react to when you press the keyboard, we need to use the noteOn callback
- internal logue-sdk functions
    - e.g. osc_sinf, osc_white, osc_sawf

[2] pluck a delay line

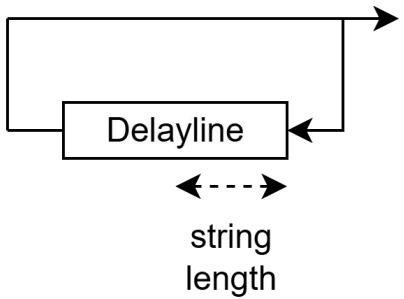
- tuning the string to a ntoe
  - delay(samples) = SampleRate / pitch(hz)
  - 48000 / 440 = 109.091 samples
- what is the value of the sample that is 109.091 samples in the past?
- read the delayline at fractional positions
  - linear interpolation



[3] linear interpolation -> lagrange interpolation

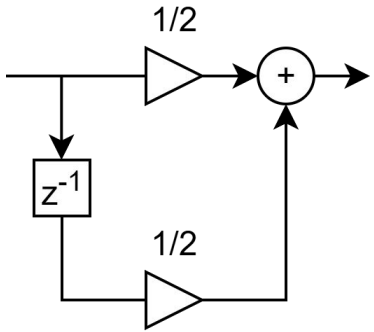
linear interpolation has inherent high frequency attenuation

juce_Delayline

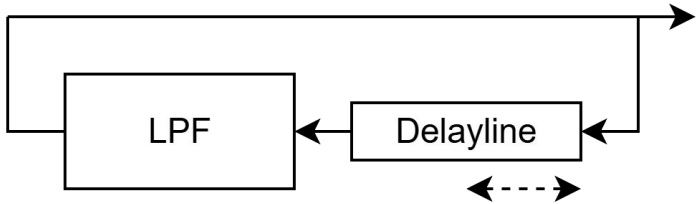


[4] 2 point average filter

- filter used in original KS paper
- add an additional 0.5 sample delay to the loop



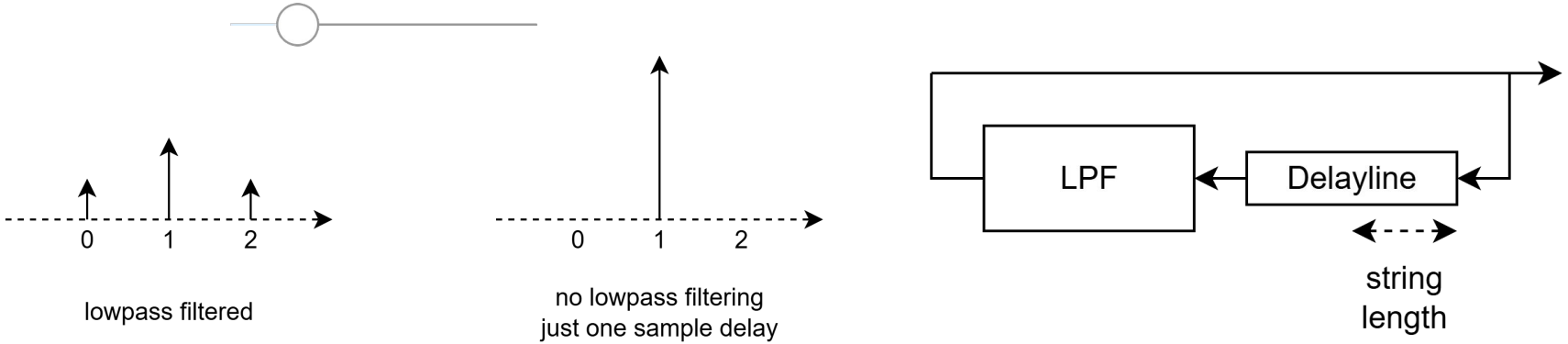
2-point average
FIR filter



string
length

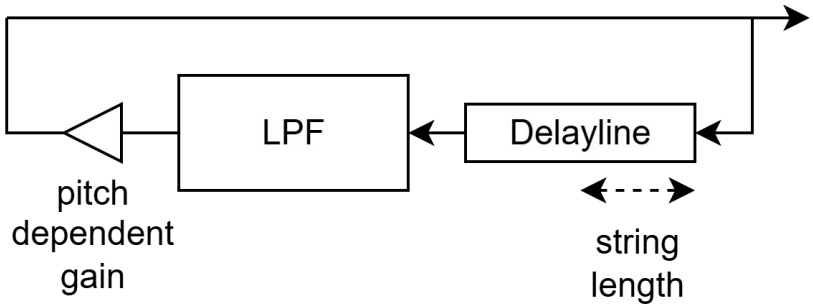
[5] 3-tap FIR damping filter (adjustable damp)

- adjustable amount of lowpass filtering
- linear phase, always add an additional 1 sample delay to the loop



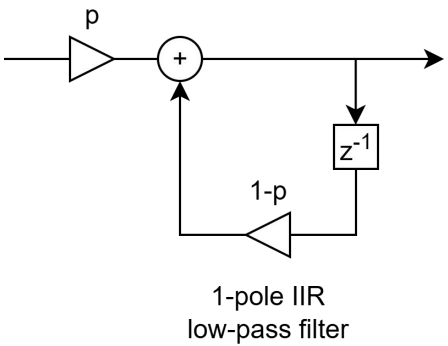
[6] pitch invariant decay time

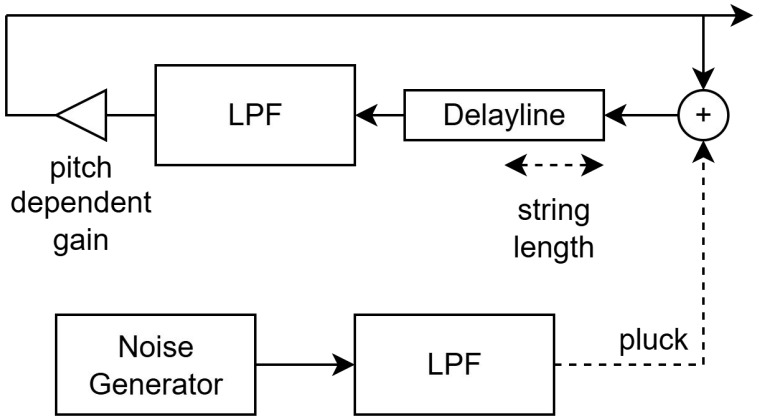
- decay time is dependent on pitch in previous example
- adjust loop gain so that all pitch have roughly the same decay time (RT60)



[7] one-pole filtering on noise input

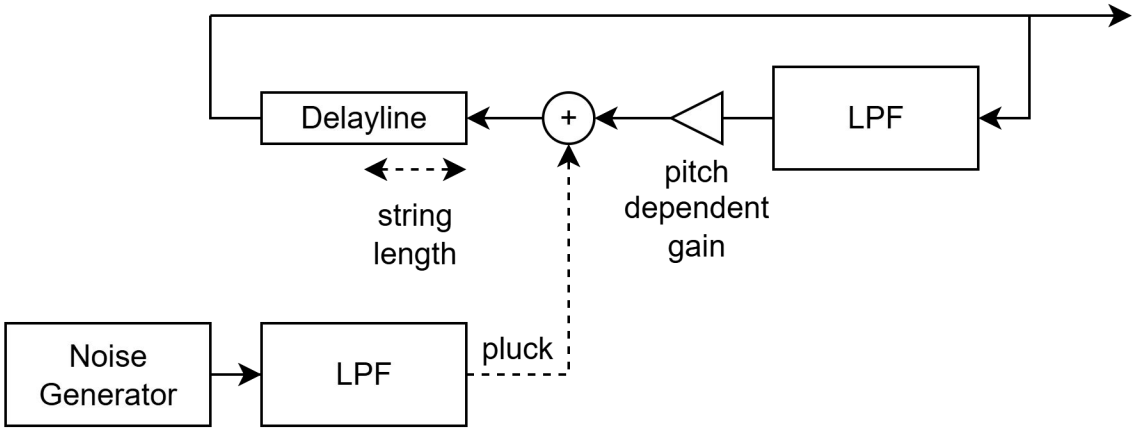
- LPF on input noise is equivalent to LPF on the output
  - because the system is LTI
- a popular technique in KS synthesis is **commuted synthesis**
  - convolve noise with the impulse response of a guitar body, store it as a wavetable
  - use the wavetable to pluck the delayline





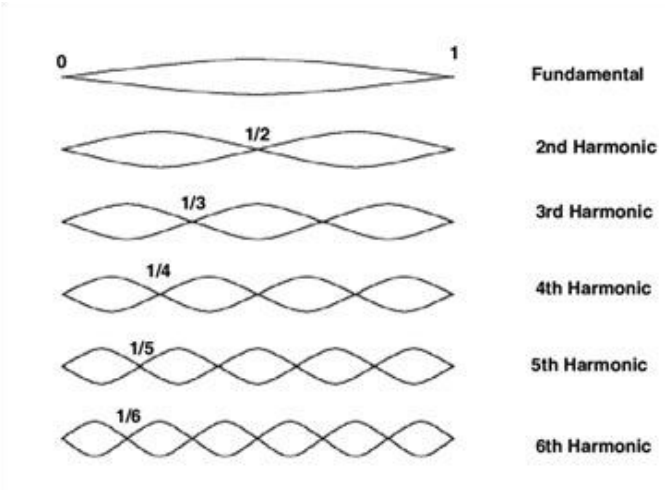
[8] reorder the loop

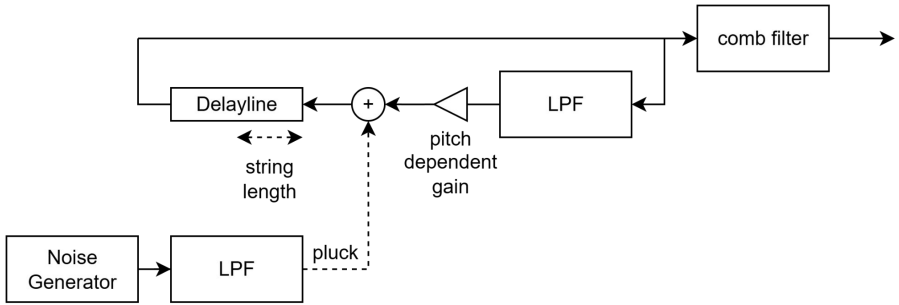
- the initial noise burst do not go through LPF on the first loop
- brighter attack



[9] pickup position (comb filter)

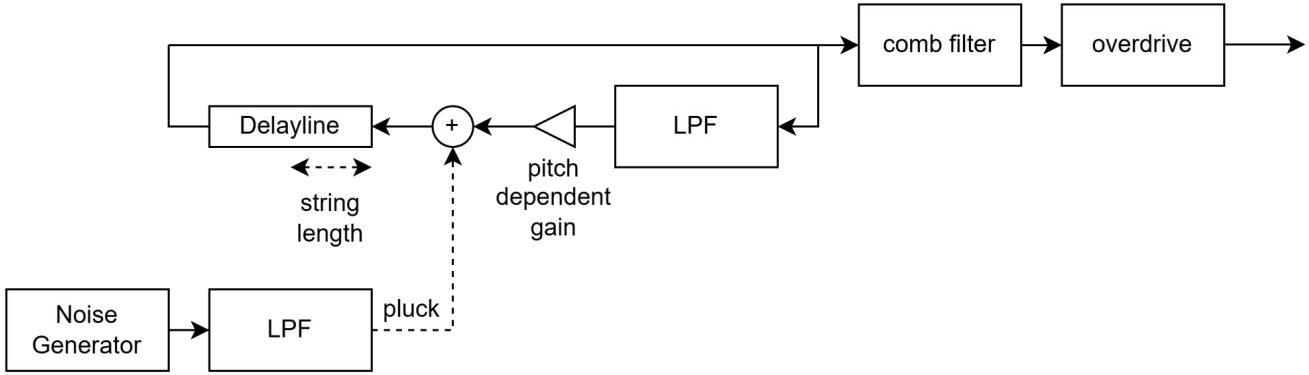
- feed forward comb filter
  - y[n] = x[n] - x[n − D]
- (very loose) emulation of variable pickup position on a string



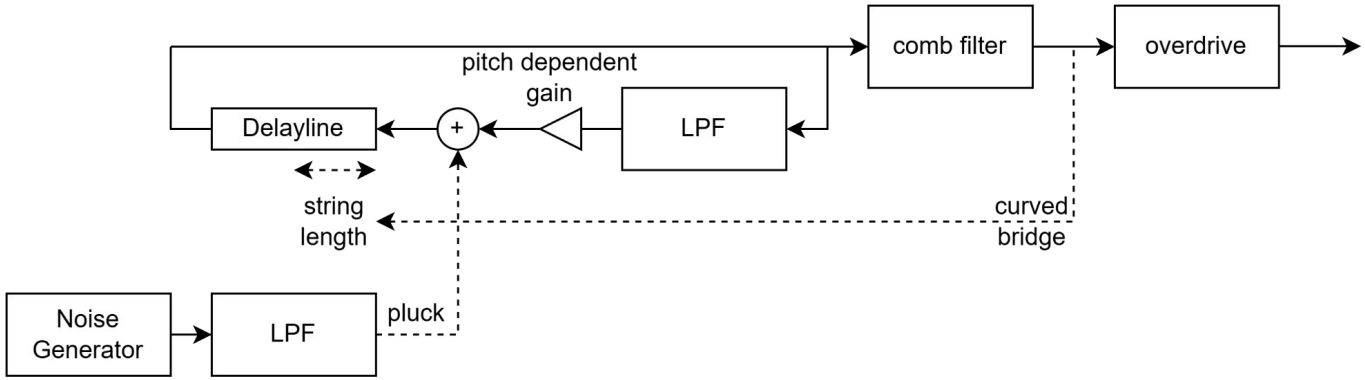


[10] overdrive

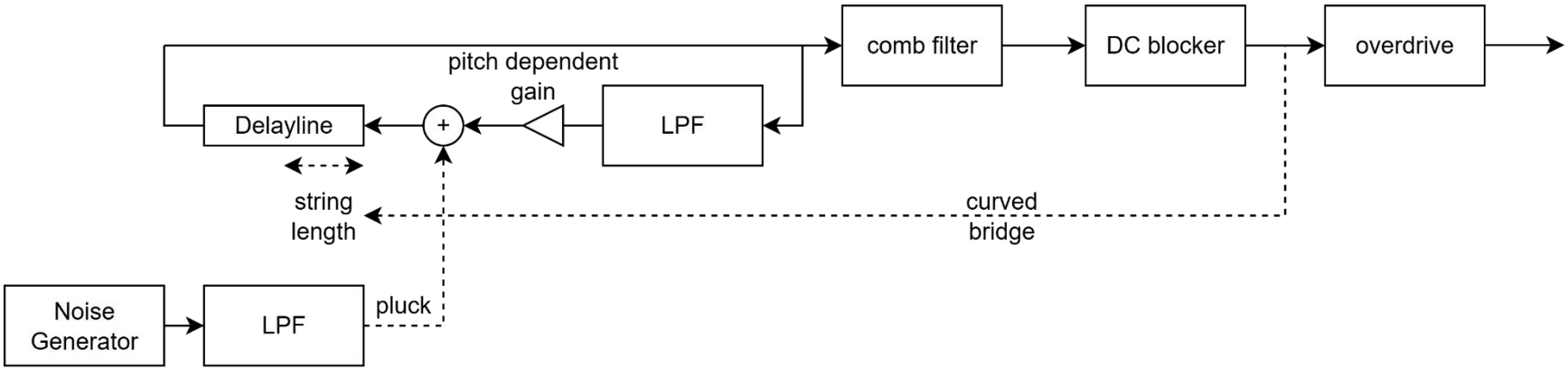
https://www.desmos.com/calculator/merzdbysff



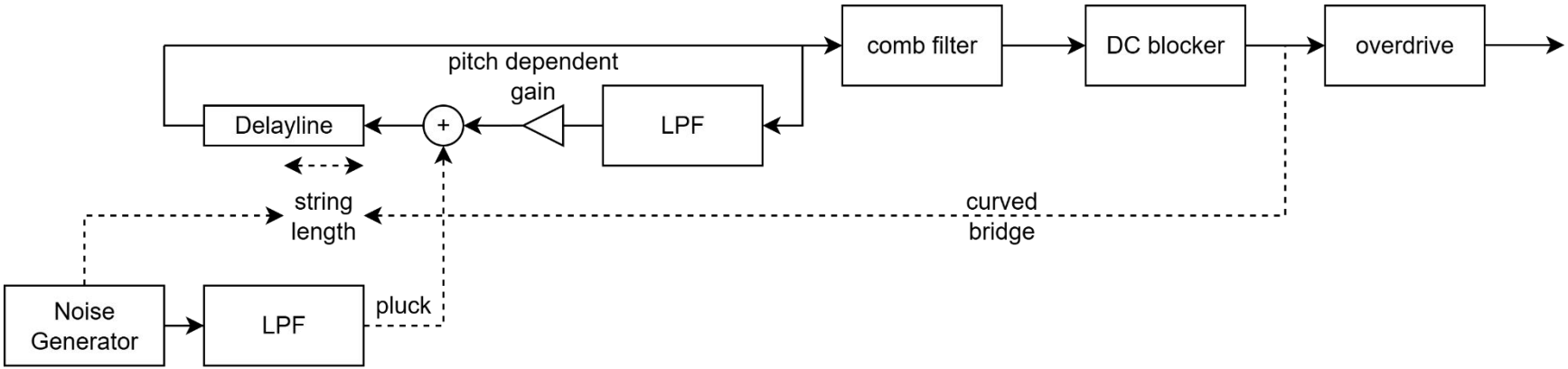
[11] curved bridge



[12] DC blocker

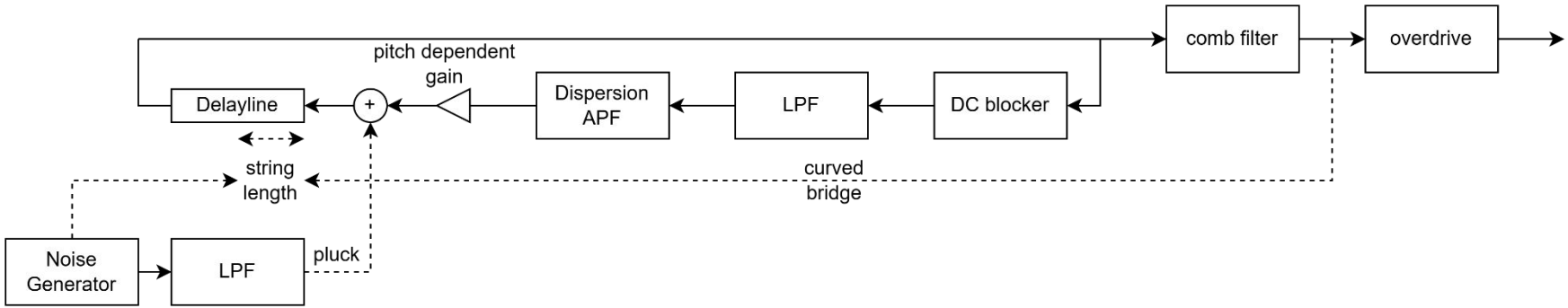


[13] FM by noise



[14] dispersion

- dispersion can cause some frequencies to be unstable
- move DC blocker inside the loop



Demos using NTS-1 and NTS-3

- Korg Kontrol Editor

Hands-on session

- feel free to modify the code and experiment with parameter mappings
- one note for Windows user: using the Docker-based build tool is recommended

NTS-3 bonus

- use sdram to get more polyphony
- can use the tuned string to process external audio input (playable resonators)
- parameter mapping to X, Y and depth control is freely assignable
- custom playing technique can be implemented using touchEvent callback

Parameter mapping

The importance of parameter mapping in electronic instrument design

one of the most cited papers of NIME

Several years ago I was invited to test out some final university projects in their prototype form in the lab. One of them was a recreation of a Theremin with modern electronic circuitry. What was particularly unusual about this was that **a wiring mistake by the student meant that the 'volume' antenna only worked when your hand was moving**. In other words the sound was only heard when there was a rate-of-change of position, rather than the traditional position-only control. It was unexpectedly exciting to play. The volume hand needed to keep moving back and forth, rather like bowing an invisible violin. I noted the effect that this had on myself and the other impromptu players in the room. **Because of the need to keep moving, it felt as if your own energy was directly responsible for the sound**. When you stopped, it stopped. The subtleties of the bowing movement gave a complex texture to the amplitude. **We were 'hooked'**. It took rather a long time to prise each person away from the instrument, as it was so engaging. I returned in a week's time and noted the irony that the 'mistake' had been corrected, deleted from the student's notes, and the traditional form of the instrument implemented.

Reference

[1] Karjalainen M, Välimäki V, Tolonen T. Plucked-string models: From the Karplus-Strong algorithm to digital waveguides and beyond[J]. Computer Music Journal, 1998, 22(3): 17-32.

[2] Jaffe D A, Smith J O. Extensions of the Karplus-Strong plucked-string algorithm[J]. Computer Music Journal, 1983, 7(2): 56-69.

[3] Emille Gillet, Mutable Instruments Rings source code, https://github.com/pichenettes/eurorack, accessed 2026/5/30

[4] Hunt A, Wanderley M M, Paradis M. The importance of parameter mapping in electronic instrument design[J]. Journal of New Music Research, 2003, 32(4): 429-440.

Q&A with Davis

- feel free to ask about any general questions about DSP programming